Reviewer Pack — Minimal Rank of Configuration Spaces vs Tree-Width

Entry 91a3c656295e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 07:56:28 UTC

Minimal Rank of Configuration Spaces vs Tree-Width

Entry ID: `91a3c656295e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 07:56:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Configuration Space Theory) **Field B** (complexity object): Complexity Theory: Monotone Circuit Depth for k-CLIQUE

Statement:

> ['For any given k-CLIQUE instance, the minimal rank of the configuration space associated with the instance is upper-bounded by a function of the tree-width of the instance.', 'Formally, there exists a constant c(k) such that for all k-CLIQUE instances G, rank(config_space(G)) ≤ c(k) * tw(G), where tw(G) is the tree-width of G.', 'The configuration space associated with an instance G is defined as the space of all configurations of the vertices and edges in G.']

Rationale (proposer's reasoning):

> ['Configuration spaces have been studied in algebraic topology for their geometric and topological properties, which may provide insights into the complexity of computing functions on graphs.', "Tree-width, a measure of the 'tree-like' structure of a graph, has been used to derive circuit lower bounds in complexity theory.", 'A connection between configuration spaces and tree-width could provide a novel approach for proving monotone circuit lower bounds.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `92128b69ebf61a15`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all k-CLIQUE instances, the ratio of the minimal rank to tree-width is ≤ c(k) + 0.1 across at least 80% of seeds and mean difference between ranks and tree-width ratios is ≤ 2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

# Helper functions for Gaussian elimination and matrix operations
def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rref = [row[:] for row in matrix]

    for i in range(rows):
        # Find pivot
        max_row = i
        for j in range(i+1, rows):
            if abs(rref[j][i]) > abs(rref[max_row][i]):
                max_row = j

        # Swap rows
        rref[i], rref[max_row] = rref[max_row], rref[i]

        # Eliminate below pivot
        for j in range(i+1, rows):
            factor = -rref[j][i] / rref[i][i]
            for k in range(cols):
                rref[j][k] += factor * rref[i][k]

    return rref

def rank(matrix):
    rref = gaussian_elimination(matrix)
    rank = 0
    for row in rref:
        if any(row):
            rank += 1
```

```
    return rank

# Function to generate a random k-clique graph
def generate_k_clique(n, k):
    G = [[0]*n for _ in range(n)]
    nodes = list(range(n))
    random.shuffle(nodes)
    clique_nodes = nodes[:k]
    for u in clique_nodes:
        for v in clique_nodes:
            if u < v:
                G[u][v] = G[v][u] = 1
    return G

# Function to compute tree-width (simplified version using BFS)
def tree_width(G):
    n = len(G)
    degree = [sum(row) for row in G]
    leaves = [i for i, d in enumerate(degree) if d == 1]

    while leaves:
        leaf = leaves.pop()
        for neighbor in range(n):
            if G[leaf][neighbor]:
                G[leaf][neighbor] = G[neighbor][leaf] = 0
                degree[neighbor] -= 1
                if degree[neighbor] == 1:
                    leaves.append(neighbor)

    return n - len(leaves)

# Main function to run a single trial
def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    k = random.randint(3, min(n-1, 6))
    G = generate_k_clique(n, k)

    config_space_rank = rank(G)
    tw = tree_width(G)

    if tw == 0:
        return {
            "metric_name": "rank/tw_ratio",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "tree-width is zero, undefined for this conjecture"
        }

    ratio = Fraction(config_space_rank, tw)
    return {
        "metric_name": "rank/tw_ratio",
        "metric_value": float(ratio),
        "instances_tested": 1,
        "conjecture_holds": True if ratio <= 2 else False,
```

```
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30))  # Default to first 29 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    conjecture_holds = all(r["conjecture_holds"] for r in results)

    if conjecture_holds:
        mean = sum(metric_values) / len(metric_values)
        std_dev = math.sqrt(sum((x - mean)**2 for x in metric_values) / len(metric_values))
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"rank/tw_ratio > 2\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_adac2c80.py", line 112, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_adac2c80.py", line 85, in run_trial
    config_space_rank = rank(G)
                        ^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_adac2c80.py", line 42, in rank
    rref = gaussian_elimination(matrix)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_adac2c80.py", line 35, in gaussian_elim
    factor = -rref[j][i] / rref[i][i]
             ~~~~~~~~~~~~^~~~~~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to provide evidence for the conjecture. | next: Re-run the test without crashing and ensure it completes successfully to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13797
2	preregistration	ollama_remote	glm4:latest	0	0	14866
3	novelty	ollama_remote	glm4:latest	0	0	8672
4	novelty	ollama_remote	glm4:latest	0	0	8558
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18678
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15310
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10598
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11572
9	judge	ollama_remote	glm4:latest	0	0	16604

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 118655 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/91a3c656295e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/91a3c656295e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/91a3c656295e.tar.gz` (if generated)